

오늘 뭐 먹지? 메뉴 추천 REST API

날씨 · 기분 · 가격대 · 함께 먹는 사람에 따라 메뉴를 추천하는 API를
Spring Boot로 직접 구현하고 문서화 · 테스트 · 컨테이너 실행까지 확장한 기록

과제 백엔드 2일차 실습 — 메뉴 추천 API 직접 만들기

제출자 광주 4반 · G122 박영서

제출일 2026년 8월 4일

저장소 `springboot-menu-recommendation`

환경 Java 21 · Spring Boot 3.4.5 · Gradle 8.14.5

실행 화면 `localhost:8080` · Swagger `localhost:8080/swagger-ui.html`

완료 필수 API 4종 Swagger 문서화 MockMvc 10종 Docker 실행

목차

01 개요

구현 범위 · 개발 환경 · 프로젝트 구조

02 주소 설계와 계층 분리

기존 매핑과의 충돌 회피 · Controller / Service 역할

03 Swagger 전체 화면

OpenApiConfig · 등록된 API 9종

04 필수 과제 ①~④

날씨별 · 기분별(JSON) · 가격대별 · 나만의 추천

05 차별화 기능

400 처리 · 문장 다듬기 · 화면 확장 · 문서화

06 자동 테스트

MockMvc 10종 · 실행 결과

07 Docker

멀티 스테이지 빌드 · 비root 실행 · 실행 검증

08 트러블슈팅 기록

막혔던 세 지점과 해결 과정

09 마무리

배운 것 · 충족도 · 더 해볼 것

1. 개요

시작 화면의 **주황색 버튼 4개**에 해당하는 API를 직접 구현하는 과제입니다. 작업 파일은 **MenuService** (추천 로직)와 **MenuController** (주소 매핑) 두 개이며, 화면(**index.html**)도 함께 확장했습니다.

여기에 더해 Swagger 문서화 · 자동 테스트 · Docker 컨테이너 실행까지 붙여 "실습용 코드"에서 끝나지 않도록 구성했습니다.

구현 범위

구분	기능	엔드포인트	학습 포인트
필수 ①	날씨별 추천	<code>GET /api/menu/weather/{weather}</code>	<code>@PathVariable</code> · 문자열 응답
필수 ②	기분별 추천	<code>GET /api/menu/mood/{mood}</code>	JSON 응답 (DTO 반환)
필수 ③	가격대별 추천	<code>GET /api/menu/price/search</code>	<code>@RequestParam</code> · 입력 검증
필수 ④	나만의 추천	<code>GET /api/menu/my/{companion}</code>	추천 기준 직접 설계
차별화	Swagger 문서화	<code>GET /swagger-ui.html</code>	<code>@Tag</code> · <code>@Operation</code> · <code>@Parameter</code>
차별화	자동 테스트	<code>./gradlew test</code>	MockMvc 10종 검증
차별화	컨테이너 실행	<code>docker run -p 8080:8080</code>	멀티 스테이지 빌드 · 비root 실행

개발 환경

구분	사용 기술	역할
언어 / 런타임	Java 21 (Eclipse Temurin)	애플리케이션 및 Docker 런타임
웹	Spring Boot 3.4.5 (Spring MVC)	내장 Tomcat, 요청 매핑, JSON 직렬화
API 문서	springdoc-openapi 2.8.13	Swagger UI · <code>/v3/api-docs</code>
테스트	JUnit 5 + MockMvc + AssertJ	상태 코드 · 응답 본문 자동 검증
빌드 / 배포	Gradle 8.14.5 + Docker	bootJar, 컨테이너 이미지 생성

프로젝트 구조

```
springboot-menu-recommendation/
├─ build.gradle
├─ Dockerfile
├─ .dockerignore
└─ src/
    ├─ main/java/com/example/menu/
    │   ├─ MenuApplication.java
    │   ├─ config/OpenApiConfig.java
    │   ├─ controller/MenuController.java
    │   ├─ dto/MenuResponse.java
    │   └─ service/MenuService.java
    ├─ main/resources/static/index.html
    └─ test/java/com/example/menu/
        └─ MenuApplicationTests.java
```

← 신규: 멀티 스테이지 빌드
← 신규
← 신규: Swagger 문서 정보
← 수정: API 4개 추가
← 수정: 추천 로직 추가
← 수정: 화면 확장
← 수정: MockMvc 10종

2. 주소 설계와 계층 분리

이번에 추가한 코드는 예외 없이 두 군데로 나뉘어 들어갔습니다.

브라우저 / Swagger → MenuController → MenuService
(주소 해석 · 문장 조립) (메뉴를 고르는 규칙)

`MenuService` 는 HTTP도 주소도 JSON도 모른 채 **메뉴 이름만** 돌려주고, `MenuController` 가 그것으로 **문장이나 JSON을** 조립합니다. 예를 들어 `recommendByWeather("rainy")` 는 **"칼국수"** 만 반환하고, **"rainy 날씨에 어울리는 메뉴는 칼국수입니다."** 라는 문장은 Controller가 만듭니다.

주소가 겹치지 않게 설계하기

기존에 `@GetMapping("/menu/{category}")` 가 이미 있었고, 이 매핑은 `/menu` 뒤의 **한 칸**을 전부 받아 갑니다. 그래서 가격 검색 주소를 `/menu/price` 로 만들면 어느 메서드가 처리할지 모호해집니다.

주소	/menu 뒤 칸 수	충돌 여부
<code>/menu/{category}</code>	1칸	기존 매핑 (기준)
<code>/menu/weather/{weather}</code>	2칸	겹치지 않음
<code>/menu/mood/{mood}</code>	2칸	겹치지 않음
<code>/menu/price/search</code>	2칸	겹치지 않음 (그래서 <code>/search</code> 를 붙임)
<code>/menu/my/{companion}</code>	2칸	겹치지 않음

추가한 4개는 모두 두 칸이고, 첫 칸 리터럴(`weather` · `mood` · `price` · `my`)이 서로 다르며 기존 `json` 과도 달라 서로 간에도 충돌하지 않습니다.

매핑이 충돌하면 앱이 **시작할 때 Ambiguous mapping** 오류로 죽습니다. `./gradlew test` 의 컨텍스트 로딩 테스트가 이를 자동으로 잡아 줍니다.

클래스 레벨 경로

`MenuController`에는 `@RequestMapping("/api")`가 붙어 있어, 메서드의 `@GetMapping`에는 `/api`를 빼고 적습니다.

```
@RequestMapping("/api") + @GetMapping("/menu/weather/{weather}")
                        = /api/menu/weather/{weather}
```

3. Swagger 전체 화면

문서 상단의 제목과 설명은 `OpenApiConfig`에서 직접 지정했습니다. springdoc이 Controller를 읽어 엔드포인트 목록은 자동으로 만들어 주지만, 문서 제목은 직접 넣어야 합니다.

```
@Configuration
public class OpenApiConfig {
    @Bean
    public OpenAPI menuOpenAPI() {
        return new OpenAPI().info(new Info()
            .title("오늘 뭐 먹지? - 메뉴 추천 API")
            .version("v1.0.0")
            .description("""
                Spring Boot 실습용 메뉴 추천 REST API입니다.
                - 실습과제 ①~④ (날씨 · 기분 · 가격대 · 함께 먹는 사람)
                - ②는 JSON(MenuResponse), 나머지는 문자열로 응답합니다.
                - 가격대별 추천은 min > max인 경우 400 Bad Request를 반환합니다.
                """));
    }
}
```


/v3/api-docs
Explore

오늘 뭐 먹지? — 메뉴 추천 API v1.0.0 OAS 3.1

[/v3/api-docs](#)

Spring Boot 실습용 메뉴 추천 REST API입니다.

- 실습과제 ①~④ (날씨 · 기분 · 가격대 · 함께 먹는 사람)
- ②는 JSON(MenuResponse), 나머지는 문자열로 응답합니다.
- 가격대별 추천은 min > max인 경우 400 Bad Request를 반환합니다.

Servers

http://localhost:8080 - Generated server url

메뉴 추천

상황에 맞는 메뉴를 추천합니다.

- GET `/api/menu` 기본 추천
- GET `/api/menu/{category}` 카테고리별 추천
- GET `/api/menu/weather/{weather}` [과제①] 날씨별 추천
- GET `/api/menu/random` 랜덤 추천
- GET `/api/menu/price/search` [과제③] 가격대별 추천 (@RequestParam)
- GET `/api/menu/my/{companion}` [과제⑤] 나만의 추천 — 같이 먹는 사람 기준
- GET `/api/menu/mood/{mood}` [과제②] 기분별 추천 (JSON)
- GET `/api/menu/json/{category}` 카테고리별 추천 (JSON)
- GET `/api/hello/{name}` 인사

Schemas

MenuResponse > Expand all **object**

Swagger UI — 등록된 API 9종 전체 목록

4. 필수 과제

① 날씨별 추천

sunny / rainy / hot / cold 네 가지 날씨에 따라 다른 메뉴를 추천합니다. 정해진 값이 아니면 안내 문구를 반환합니다.

```
public String recommendByWeather(String weather) {
    return switch (weather) {
        case "sunny" -> "비빔밥";
        case "rainy" -> "칼국수";
        case "hot" -> "냉면";
        case "cold" -> "김치찌개";
        default -> NOT_FOUND;
    };
}
```

```
@GetMapping("/menu/weather/{weather}")
public String menuByWeather(@PathVariable("weather") String weather) {
    String menu = menuService.recommendByWeather(weather);

    if (MenuService.NOT_FOUND.equals(menu)) {
        return weather + "에 대한 " + menu + ". (sunny, rainy, hot, cold 중에서 골라 주세요)";
    }
    return weather + " 날씨에 어울리는 메뉴는 " + menu + "입니다.";
}
```

반환 타입이 `String` 이면 Spring이 그 문자열을 그대로 응답 본문으로 내보냅니다 (`Content-Type: text/plain; charset=UTF-8`).

rainy 날씨에 어울리는 메뉴는 칼국수입니다.

`weather/rainy` — 비 오는 날은 칼국수

hot 날씨에 어울리는 메뉴는 냉면입니다.

`weather/hot` — 값을 바꾸면 결과가 달라진다

GET
/api/menu/weather/{weather}
[과제①] 날씨별 추천

sunny / rainy / hot / cold 네 가지 날씨에 따라 다른 메뉴를 추천합니다. 정해진 값이 아니면 안내 문구를 반환합니다.

Parameters

Name

Description

weather * required

날씨 (sunny / rainy / hot / cold)

string (path)

rainy

Execute

Clear

Responses

Curl

curl -X 'GET' \
'http://localhost:8080/api/menu/weather/rainy' \
-H 'accept: */*'

Request URL

http://localhost:8080/api/menu/weather/rainy

Server response

Code

Details

200

Response body

rainy 날씨에 어울리는 메뉴는 칼국수입니다.

Response headers

connection: keep-alive
content-length: 58
content-type: text/plain; charset=UTF-8
date: Tue, 04 Aug 2026 08:06:08 GMT
keep-alive: timeout=60

Responses

Code

Description

Links

200

OK

Media type

/

Controls Accept header.

Example Value

Schema

string

No links

Swagger UI에서 직접 실행 — 200 OK

메뉴 추천 REST API · 광주 4반 G122 박영서

7

② 기분별 추천 — JSON 응답

`happy` / `sad` / `tired` / `stressed` 에 따라 추천합니다. 다른 과제와 달리 `String` 이 아닌 `MenuResponse` 객체를 반환하므로 Spring이 JSON으로 변환합니다.

```
@GetMapping("/menu/mood/{mood}")
public MenuResponse menuByMood(@PathVariable("mood") String mood) {
    String menu = menuService.recommendByMood(mood);

    return new MenuResponse(
        mood,
        menu,
        MenuService.NOT_FOUND.equals(menu)
            ? mood + "에 대한 " + menu + ". (happy, sad, tired, stressed 중에서 골라 주세요)"
            : "기분이 " + mood + "일 땐 " + menu + " 어떠세요?"
    );
}
```

`@RestController` 가 붙어 있으면 객체를 돌려주는 순간 Spring이 JSON으로 바꿔 내보냅니다. 따로 변환 코드를 쓰지 않습니다. 이 변환을 담당하는 것이 Jackson이며 `spring-boot-starter-web` 에 이미 들어 있습니다.

`MenuResponse` 는 `record` 입니다. Jackson은 record의 필드 이름을 그대로 JSON 키로 씁니다.

```
public record MenuResponse(String category, String menu, String message) { }
```

기분이 happy일 땐 치킨 어떠세요?

(원본 JSON 응답)

```
{
  "category": "happy",
  "menu": "치킨",
  "message": "기분이 happy일 땐 치킨 어떠세요?"
}
```

`mood/happy` — message를 문장으로 보여주고 원본 JSON을 함께 표시

GET
/api/menu/mood/{mood}
[과제②] 기분별 추천 (JSON)

happy / sad / tired / stressed 에 따라 추천합니다. 다른 과제와 달리 String이 아닌 MenuResponse 객체를 반환하므로 JSON으로 응답됩니다.

Parameters
Cancel

Name	Description
mood * required string (path)	기분 (happy / sad / tired / stressed) happy

Execute
Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8080/api/menu/mood/happy' \
-H 'accept: */*'

```

Request URL

```
http://localhost:8080/api/menu/mood/happy

```

Server response

Code	Details
200	Response body <pre>{ "category": "happy", "menu": "치킨", "message": "기분이 happy일 땐 치킨 어떠세요?" }</pre> Download Response headers <pre>connection: keep-alive content-type: application/json date: Tue, 04 Aug 2026 08:06:13 GMT keep-alive: timeout=60 transfer-encoding: chunked </pre>

Responses

Code	Description	Links
200	OK Media type: */ Controls Accept header. Example Value Schema <pre>{ "category": "string", "menu": "string", "message": "string" }</pre>	No links

Swagger 실행 결과 — `content-type: application/json`, 세 필드가 직렬화됨

③ 가격대별 추천 — @RequestParam

앞의 두 과제와 다른 점이 있습니다. 값을 두 개, 그것도 주소 경로가 아니라 `?min=5000&max=10000` 형태로 받습니다.

구분	주소 모양	애노테이션
경로의 일부	<code>/api/menu/mood/happy</code>	<code>@PathVariable</code>
질의 문자열	<code>/api/menu/price/search?min=5000&max=10000</code>	<code>@RequestParam</code>

```
@GetMapping("/menu/price/search")
public String menuByPrice(@RequestParam("min") int min,
                          @RequestParam("max") int max) {
    if (min > max) {
        throw new ResponseStatusException(
            HttpStatus.BAD_REQUEST,
            "min(" + min + ")은 max(" + max + ")보다 클 수 없습니다."
        );
    }
    String menu = menuService.recommendByPrice(max);
    return min + "~" + max + "원 가격대라면 " + menu + " 어떠세요?";
}
```

파라미터 타입이 `String` 이 아니라 `int` 인 점을 눈여겨봤습니다. 주소로 들어오는 값은 원래 전부 문자열인데 Spring이 `int` 로 바꿔 줍니다. 숫자가 아닌 값(`?min=abc`)이 오면 변환에 실패해 Spring이 자동으로 400을 반환합니다.

조건	추천 메뉴
<code>max ≤ 6,000원</code>	김밥
<code>max ≤ 12,000원</code>	돈가스
그 외	소고기 구이

5000~10000원 가격대라면 돈가스 어떠세요?

`min=5000&max=10000` — 12,000원 이하 구간이므로 돈가스

GET /api/menu/price/search [과제③] 가격대별 추천 (@RequestParam)

값을 경로가 아니라 쿼리 파라미터로 두 개 받습니다. max 기준으로 구간을 나눕니다. min이 max보다 크면 400 Bad Request를 반환합니다.

Parameters

min * required

최소 금액

integer(\$int32)

(query)

5000

max * required

최대 금액

integer(\$int32)

(query)

10000

Execute

Clear

Responses

Curl

curl -X 'GET' \
'http://localhost:8080/api/menu/price/search?min=5000&max=10000' \
-H 'accept: */*'

Request URL

http://localhost:8080/api/menu/price/search?min=5000&max=10000

Server response

Code

Details

200

Response body

5000~10000원 가격대라면 돈가스 어떠세요?

Download

Response headers

connection: keep-alive
content-length: 53
content-type: text/plain; charset=UTF-8
date: Tue, 04 Aug 2026 08:06:18 GMT
keep-alive: timeout=60

Responses

Code

Description

Links

200

OK

No links

Media type

/

Controls Accept header.

Example Value | Schema

string

Swagger 실행 결과 — 쿼리 파라미터 두 개가 Request URL 에 반영됨

메뉴 추천 REST API · 광주 4반 G122 박영서

11

④ 나만의 추천 — 같이 먹는 사람 기준

추천 기준을 "누구와 함께 먹는가"로 설계했습니다.

값	의미	추천 메뉴
solo	혼밥	라면
friend	친구와 함께	피자
family	가족과 함께	불고기
그 외	정해지지 않은 값	기존 randomMenu() 로 무작위 추천

```
public String recommendByCompanion(String companion) {
    return switch (companion) {
        case "solo"    -> "라면";
        case "friend"  -> "피자";
        case "family"  -> "불고기";
        default -> randomMenu(); // ← 기존 menus 리스트를 재사용
    };
}
```

앞의 세 과제와 달리 default 에서 안내 문구가 아니라 이미 만들어져 있던 randomMenu() 를 호출합니다. 메뉴 목록을 새로 만들지 않고 기존 메서드를 그대로 재사용한 것입니다.

```
@GetMapping("/menu/my/{companion}")
public String menuByCompanion(@PathVariable("companion") String companion) {
    String menu = menuService.recommendByCompanion(companion);

    return switch (companion) {
        case "solo"    -> "혼자 먹기 좋은 " + menu + " 어떠세요?";
        case "friend"  -> "친구랑 나눠 먹기 좋은 " + menu + " 어떠세요?";
        case "family"  -> "가족과 함께라면 " + menu + " 어떠세요?";
        default -> "오늘은 " + menu + " 어떠세요?";
    };
}
```

혼자 먹기 좋은 라면 어떠세요?

my/solo — 혼밥 추천

가족과 함께라면 불고기 어떠세요?

my/family — 조건에 따라 결과가 달라진다

GET

/api/menu/my/{companion} [과제④] 나만의 추천 — 같이 먹는 사람 기준

추천 기준을 '누구와 함께 먹는가'로 직접 설정했습니다. solo / friend / family 외의 값은 랜덤 추천으로 대체합니다.

Parameters

Cancel

Name	Description
companion required	같이 먹는 사람 (solo / friend / family)
string (path)	solo

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8080/api/menu/my/solo' \
-H 'accept: */*'
```

Request URL

http://localhost:8080/api/menu/my/solo

Server response

Code	Details
200	Response body 혼자 먹기 좋은 라면 어떠세요? Download Response headers <pre>connection: keep-alive content-length: 41 content-type: text/plain; charset=UTF-8 date: Tue, 04 Aug 2026 08:06:28 GMT keep-alive: timeout=60</pre>

Responses

Code	Description	Links
200	OK Media type */* Controls Accept header. Example Value Schema string	No links

Swagger 실행 결과 — my/solo

5. 차별화 기능

5-1. 잘못된 입력의 400 처리

`min=20000&max=5000` 은 서버 잘못이 아니라 요청 잘못입니다. 이럴 때 200 OK에 "잘못됐다"고 적어 보내면 프로그램은 성공과 실패를 구분할 수 없습니다. HTTP 상태 코드로 알려 주는 것이 REST의 방식입니다.

검증 조건	처리 결과
<code>min > max</code>	400 Bad Request + 안내 메시지
<code>min</code> 또는 <code>max</code> 가 숫자가 아님	400 Bad Request (Spring 자동 변환 실패)
파라미터 누락	400 Bad Request (required 기본값)
<code>min ≤ max</code>	200 정상 추천 결과 반환

설정이 하나 더 필요했습니다 — Spring Boot 2.3부터 `server.error.include-message`의 기본값이 `never`입니다. 이 상태에서는 `ResponseStatusException`에 적은 안내 문구가 응답 본문에서 **빠진 채** 상태 코드만 나갑니다.

```
server.error.include-message=always
```

```
// 설정 전 (기본값 never)
{"timestamp":"...", "status":400, "error":"Bad Request", "path":"/api/menu/price/search"}

// 설정 후 (always)
{"timestamp":"...", "status":400, "error":"Bad Request",
 "message":"min(20000)은 max(5000)보다 클 수 없습니다.", "path":"/api/menu/price/search"}
```

GET /api/menu/price/search [과제③] 가격대별 추천 (@RequestParam)

값을 경로나 아니라 쿼리 파라미터로 두 개 받습니다. max 기준으로 구간을 나눕니다. min이 max보다 크면 400 Bad Request를 반환합니다.

Parameters

Name	Description
min * required integer(\$int32) (query)	최소 금액 20000
max * required integer(\$int32) (query)	최대 금액 5000

Execute Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:8080/api/menu/price/search?min=20000&max=5000' \
-H 'accept: */*'
```

Request URL

```
http://localhost:8080/api/menu/price/search?min=20000&max=5000
```

Server response

Code	Details
400 <i>Undocumented</i>	Error: response status is 400 Response body <pre>{ "timestamp": "2026-08-04T08:06:23.682+00:00", "status": 400, "error": "Bad Request", "message": "min(20000)은 max(5000)보다 클 수 없습니다.", "path": "/api/menu/price/search" }</pre> Download Response headers <pre>connection: close content-type: application/json date: Tue, 04 Aug 2026 08:06:23 GMT transfer-encoding: chunked</pre>

Responses

Code	Description	Links
200	OK Media type */* Controls Accept header. Example Value Schema string	No links

min=20000, max=5000 — 400 Bad Request와 안내 메시지가 함께 반환됨

오류: HTTP 400

요청: GET /api/menu/price/search?min=20000&max=5000

min(20000)은 max(5000)보다 클 수 없습니다.

(응답 본문)

```
{
  "timestamp": "2026-08-04T08:07:20.606+00:00",
  "status": 400,
  "error": "Bad Request",
  "message": "min(20000)은 max(5000)보다 클 수 없습니다.",
  "path": "/api/menu/price/search"
}
```

화면에서도 상태 코드와 응답 본문을 함께 표시하도록 구현

5-2. 응답 문장 다듬기

처음 구현했을 때 없는 값을 넣으면 이런 응답이 나왔습니다.

없는날씨 날씨에 어울리는 메뉴는 추천 가능한 메뉴가 없습니다입니다.

"...없습니다" 와 "입니다." 가 이어 붙어 말이 되지 않습니다. 문장을 조립하기 전에 "메뉴를 찾았는지"를 먼저 확인해야 했습니다. 그래서 실패 문구를 `MenuService` 의 상수로 분리했습니다.

```
public static final String NOT_FOUND = "추천 가능한 메뉴가 없습니다";
```

- 실패 문구 일원화 — 같은 문자열을 Service와 Controller 두 곳에 각각 적어 두면, 나중에 문구를 고칠 때 한쪽만 고치는 실수가 납니다. 그러면 `if` 조건이 조용히 어긋납니다.
- `NOT_FOUND.equals(menu)` 순서 — `menu` 가 `null` 이면 `menu.equals(...)` 는 `NullPointerException` 이 납니다. 확실히 `null` 이 아닌 쪽을 앞에 두었습니다.

snowy에 대한 추천 가능한 메뉴가 없습니다. (sunny, rainy, hot, cold 중에서 골라 주세요)

없는 값을 넣어도 문장이 자연스럽게, 사용 가능한 값을 함께 안내한다

5-3. 화면(index.html) 확장

기본 제공 화면은 과제별 버튼이 하나씩뿐이라 값을 바꿔 볼 수 없었습니다. 다음을 추가했습니다.

- 값 변형 버튼 — 각 과제의 다른 입력값(맑은 날 · 더운 날 · 추운 날, 가격 3구간, 동행 3종)
- 실패 케이스 버튼 — 없는 값(`snowy` · `angry` · `pet`)과 잘못된 가격 범위를 회색 버튼으로 구분
- JSON 응답 표시 개선 — `message` 를 문장으로 먼저 보여주고 원본 JSON을 함께 표시
- 오류 본문 표시 — 기존 코드는 `!response.ok` 에서 바로 예외를 던져 `오류: HTTP 400` 만 보였습니다. 상태 코드와 응답 본문을 함께 보여주도록 수정


```
// 400 같은 실패 응답도 본문을 그대로 보여준다.  
if (!response.ok) {  
  const reason = isJson && body.message ? body.message : "";  
  result.textContent =  
    `오류: HTTP ${response.status} ${response.statusText}\n`  
    + `요청: GET ${url}\n\n`  
    + (reason ? `${reason}\n\n` : "")  
    + (isJson ? `(응답 본문)\n${JSON.stringify(body, null, 2)}` : body);  
  return;  
}
```

🍴 오늘 뭐 먹지?

버튼을 누르면 Spring Boot REST API의 응답을 확인할 수 있습니다.

기본 기능

이미 만들어져 있는 API입니다. 눌러서 동작을 확인해 보세요.

기본 추천

랜덤 추천

한식 추천

중식 추천

일식 추천

JSON 응답

실습과제 직접 구현

MenuService(추천 규칙)와 MenuController(주소 매핑)에 직접 작성한 API입니다. 값을 바꿔 가며 눌러 결과가 어떻게 달라지는지 확인해 보세요.

① 날씨별 추천

② 기분별 추천

③ 가격대별 추천

④ 나만의 추천

GET ① 날씨별 — 값 바꿔 보기 (문자열 응답)

맑은 날

더운 날

추운 날

없는 값 (snowy)

GET ② 기분별 — JSON 응답

우울한 날

지친 날

스트레스

없는 값 (angry)

GET ③ 가격대별 — 쿼리 파라미터

6,000원 이하

6,000~12,000원

12,000원 이상

잘못된 범위 (min > max)

GET ④ 나만의 추천 — 같이 먹는 사람

친구와

가족과

없는 값 (pet)

결과가 여기에 표시됩니다.

확장한 통합 테스트 화면 — 과제별 값 변형 버튼과 실패 케이스 버튼

5-4. Swagger 문서화

엔드포인트 목록만 나오는 기본 상태에서, 각 API의 목적과 입력 예시까지 문서에 담았습니다.

```

@Tag(name = "메뉴 추천", description = "상황에 맞는 메뉴를 추천합니다.")
public class MenuController {

    @Operation(summary = "[과제③] 가격대별 추천 (@RequestParam)",
        description = "값을 경로가 아니라 쿼리 파라미터로 두 개 받습니다. ...")
    @GetMapping("/menu/price/search")
    public String menuByPrice(
        @Parameter(description = "최소 금액", example = "5000") @RequestParam("min") int min,
        @Parameter(description = "최대 금액", example = "10000") @RequestParam("max") int max)
    { ... }
}

```

`example` 을 지정해 두면 Swagger UI에서 Try it out을 눌렀을 때 입력칸이 예시 값으로 미리 채워집니다. 앞의 실행 화면들이 모두 그 상태입니다.

6. 자동 테스트

MockMvc로 상태 코드 · 응답 본문 · JSON 경로를 자동 검증했습니다. 서버를 직접 띄우지 않고 `DispatcherServlet` 에 가짜 요청을 보내 응답을 확인합니다. 화면을 눌러 보는 수동 확인과 달리, 코드를 고친 뒤에도 같은 검증을 반복할 수 있습니다.

```

@SpringBootTest
@AutoConfigureMockMvc
class MenuApplicationTests {

    @Autowired
    private MockMvc mockMvc;

    @Test
    @DisplayName("[과제②] 기분별 추천 - MenuResponse가 JSON 세 필드로 직렬화된다")
    void moodApiReturnsJson() throws Exception {
        mockMvc.perform(get("/api/menu/mood/happy"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.category").value("happy"))
            .andExpect(jsonPath("$.menu").value("치킨"))
            .andExpect(jsonPath("$.message").exists());
    }
}

```

테스트 메서드	검증 내용
<code>contextLoads</code>	Spring Container 기동 · 매핑 충돌 여부
<code>weatherApiReturnsDifferentMenus</code>	과제① 날씨에 따라 다른 문자열 반환
<code>weatherApiGuidesOnUnknownValue</code>	과제① 정해지지 않은 값의 안내 문구
<code>moodApiReturnsJson</code>	과제② JSON 세 필드 직렬화
<code>priceApiSplitsByMax</code>	과제③ 가격 3구간이 각각 다른 메뉴 반환
<code>priceApiRejectsInvalidRange</code>	과제③ <code>min > max</code> → 400
<code>priceApiRejectsNonNumericInput</code>	과제③ 숫자가 아닌 입력 → 400
<code>companionApiReturnsPersonalizedMenu</code>	과제④ 동행별 결과 분기
<code>existingApisStillWork</code>	기존 API 회귀 확인
<code>openApiDocumentationExposesAssignmentApis</code>	OpenAPI 문서에 과제 API 4종 노출

```
$ ./gradlew clean test
```

```
> Task :clean
> Task :compileJava
> Task :compileTestJava
> Task :test
```

```
BUILD SUCCESSFUL in 2s
5 actionable tasks: 5 executed
```

```
총 10개 / 실패 0 / 건너뛴 0 / 0.47초
```

7. Docker

멀티 스테이지 빌드로 빌드 환경과 실행 환경을 분리했습니다. 빌드에는 JDK가 필요하지만 실행에는 JRE면 충분하므로, 최종 이 미지에는 컴파일러를 넣지 않습니다.

```

FROM eclipse-temurin:21-jdk AS builder          # 1단계: jar 생성
WORKDIR /build
COPY gradlew ./
COPY gradle gradle
COPY build.gradle settings.gradle ./
RUN chmod +x gradlew && ./gradlew dependencies --no-daemon
COPY src src
RUN ./gradlew bootJar --no-daemon -x test

FROM eclipse-temurin:21-jre                    # 2단계: 실행만
WORKDIR /app
RUN useradd --create-home --shell /bin/bash appuser
USER appuser
COPY --from=builder /build/build/libs/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]

```

그 외 적용한 것:

- 레이어 캐시 최적화 — 의존성 파일을 소스보다 먼저 복사해, 소스만 고쳤을 때 재빌드가 빠르도록 구성
- 비root 실행 — `appuser` 계정으로 실행
- `.dockerignore` — `build/`, `.gradle/`, `bin/`, `.git/` 제외

```

$ docker build -t menu-recommendation:latest .
=> naming to docker.io/library/menu-recommendation:latest   done

$ docker images menu-recommendation
REPOSITORY          TAG          SIZE
menu-recommendation latest       567MB

$ docker run -d --name menu-app -p 8080:8080 menu-recommendation:latest
$ docker ps
NAMES          IMAGE                                STATUS          PORTS
menu-app       menu-recommendation:latest          Up 7 minutes    0.0.0.0:8080->8080/tcp

$ docker logs menu-app
INFO 1 --- [main] com.example.menu.MenuApplication
      : Starting MenuApplication v0.0.1-SNAPSHOT using Java 21.0.11
      with PID 1 (/app/app.jar started by appuser in /app)
INFO 1 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer
      : Tomcat started on port 8080 (http) with context path '/'
INFO 1 --- [main] com.example.menu.MenuApplication
      : Started MenuApplication in 0.733 seconds

```

이 보고서의 모든 Swagger · 화면 캡처는 위 Docker 컨테이너 (`http://localhost:8080`)에서 실행한 결과입니다. 즉 이 미지 빌드부터 API 동작까지 하나의 흐름으로 검증되었습니다.

8. 트러블슈팅 기록

① 400 응답에 메시지가 실리지 않음

`ResponseStatusException` 에 안내 문구를 적었는데 응답 본문에 나타나지 않았습니다. Spring Boot 2.3부터 `server.error.include-message` 의 기본값이 `never` 이기 때문이었고, `always` 로 바꿔 해결했습니다. 에러 메시지에 내부 정보가 새어 나가는 것을 막기 위한 안전 장치이므로, 실제 서비스에서는 켜지 말지 신중히 판단해야 한다는 점도 함께 알게 되었습니다.

② MockMvc에서 400 응답 본문이 비어 있음

`jsonPath("$.message")` 검증이 실패했습니다. MockMvc는 서블릿 컨테이너의 `/error` 포워딩을 거치지 않아 에러 본문이 생성되지 않았습니다. 상태 코드와 함께 실제로 던져진 예외와 그 사유를 검증하는 방식으로 바꾸었고, 본문에 메시지가 실리는지는 실행 중인 서버에서 따로 확인했습니다.

```
.andExpect(status().isBadRequest())
.andExpect(result -> {
    Exception thrown = result.getResolvedException();
    assertThat(thrown).isInstanceOf(ResponseStatusException.class);
    assertThat(((ResponseStatusException) thrown).getReason())
        .contains("min(20000)", "max(5000)", "클 수 없습니다");
});
```

③ 안내 문구가 어색하게 이어 붙음

`"...없습니다" + "입니다."` 형태로 문장이 겹쳤습니다. 실패 문구를 `NOT_FOUND` 상수로 분리하고, 문장을 조립하기 전에 분기하도록 수정했습니다.

9. 마무리

배운 것

- `@PathVariable` 과 `@RequestParam` 의 적용 상황을 구분하고 실제 엔드포인트에 사용했다.
- Controller는 요청 매핑과 응답 조립, Service는 추천 규칙이라는 계층별 책임을 코드로 분리했다.
- 반환 타입 하나로 문자열 응답과 JSON 응답이 같린다는 점을 `record` DTO와 Jackson 직렬화로 확인했다.
- 기존 `/menu/{category}` 매핑과 충돌하지 않도록 주소를 설계하는 과정을 거쳤다.
- 잘못된 입력을 HTTP 400으로 처리하고, `server.error.include-message` 설정까지 확인했다.
- Swagger · MockMvc · Docker로 API의 문서화 · 검증 · 실행 환경을 함께 완성했다.

요구사항 충족도

평가 항목	결과	근거
필수 API 4종	완료	날씨 · 기분 · 가격대 · 동행 실행 화면과 자동 테스트
API 문서화	완료	Swagger UI 및 <code>/v3/api-docs</code> , 엔드포인트 9종 노출
입력 검증	완료	<code>min > max</code> · 숫자 아님 · 파라미터 누락 3가지 모두 400
화면 확장	완료	값 변형 · 실패 케이스 버튼, 오류 본문 표시
자동 테스트	완료	MockMvc 10종, BUILD SUCCESSFUL
컨테이너 실행	완료	멀티 스테이지 Dockerfile, 8080 포트 실행

더 해볼 것

- 날씨 · 기분 값을 `enum` 으로 바꾸면 오타를 컴파일 단계에서 잡을 수 있다.
- 추천 규칙이 코드에 하드코딩되어 있으므로, `Map` 이나 설정 파일로 분리하면 코드 수정 없이 메뉴를 바꿀 수 있다.
- 현재는 조건 하나만 반영하므로, 날씨 · 기분 · 예산을 동시에 고려한 종합 추천으로 확장할 수 있다.
- 입력 검증을 조건문으로 직접 처리하고 있으므로, `spring-boot-starter-validation` 과 DTO 검증을 적용할 수 있다.
- 기존 `menuByCategory` 에는 아직 5-2의 문장 겹침 문제가 남아 있어 동일하게 정리할 수 있다.